Q: any questions?

Lecture 8: The World Wide Web and HTTP

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

Contents

- The World Wide Web and URLs
- HyperText Transfer Protocol (HTTP)
- HTML
- HTTP Servers
- Challenges for Distributed Systems & the WWW

The World Wide Web and URLs

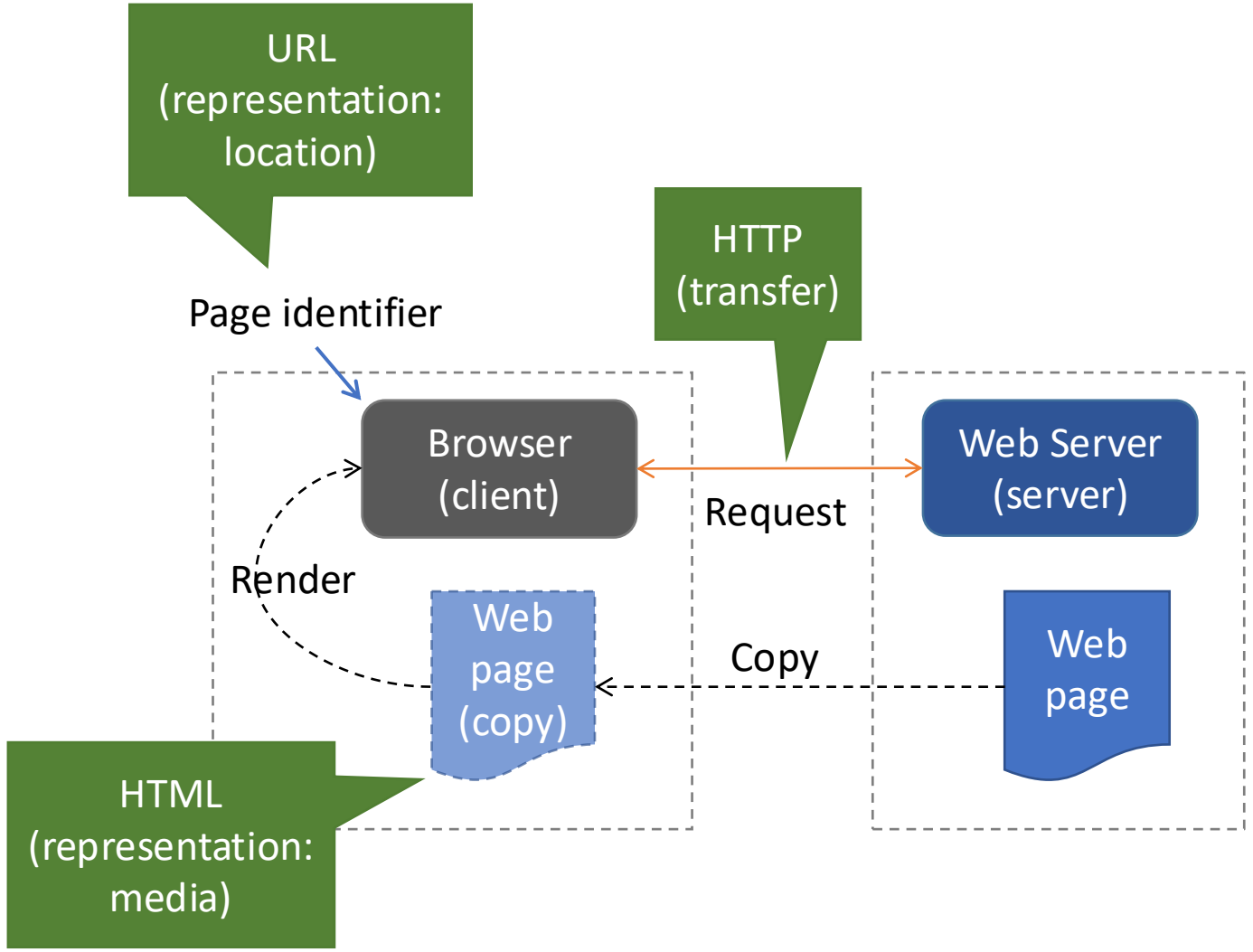
Introduction

- The world wide web is a distributed system for publishing and accessing resources across the Internet
  - It is a **client-server** system
    - Web **browsers** are the commonest (but not only) client
  - It is based on a **request-response** protocol (**HTTP**)
    - This operates over the standard Internet Protocol TCP

Web Protocols

- Web is complex
  - many **protocol** standards (i.e. rules) have been devised to specify various aspects and details
- The most important are:
  - **HyperText Transfer Protocol** (**HTTP**): a transfer protocol that specifies how a browser interacts with a web server
  - **Uniform Resource Locator** (**URL**): a representation standard that specifies the format of web page identifiers
  - **HyperText Markup Language** (**HTML**): a representation standard that specifies the contents and layout of a web page

Overview: viewing a web page



URL syntax

- The Web uses a **syntactic** form known as a **Uniform Resource Locator** (**URL**) to specify a web page
- The general form of a URL is:

  protocol://**computer_name**:**port**/**document_name**?**parameters**
- where:
  - **protocol** (or “scheme”) is the name of the protocol used to access the document
  - **computer_name** is the domain name of the computer on which the document resides (or an IP address)
  - **port** (optional) port number at which the server is listening
  - **document_name** (optional) name of the document
  - ? (optional) **parameters** for the page
  - Example: https://people.cs.nott.ac.uk/pszcmg/Project_ideas.html

URL defaults

- In a typical URL, a user can omit many of the parts, e.g.

  www.cs.nott.ac.uk

- Which omits
  - the protocol (**http** is assumed)
  - the port (**80** is assumed for http, **443** for https)
  - the document name (**/** is assumed)
    - (but many **servers** automatically try e.g. “/index.html” or similar for “/”)
  - and parameters (none are assumed)
- A URL may also be **relative** to the current page URL
  - E.g. just a document name

    details.html
  - The values for the current page are taken as a starting point (unless an explicit base URL is specified in the page)

Browser: Handling a URL

The browser [tab] (client):

1. divides the URL into protocol, computer name (and port), document name and parameters
2. Uses the protocol to decide which protocol to use, e.g. for HTTP…
3. uses the computer name and protocol port to form a TCP **connection** to the server on which the page resides
4. uses the computer name, document name and parameters to form an HTTP request
5. displays the content (or error) received from the server

HyperText Transfer Protocol (HTTP)

Web Document Transfer with HTTP

- **HyperText Transfer Protocol** (**HTTP**) is the primary transfer protocol that a browser uses to interact with a web server
  - Used for URLs which contain an explicit protocol reference of http:// or omit the protocol (HTTP is assumed)
- HTTP is a **request-response** protocol
  - **Client** sends request
  - **server** replies with response
- Human-readable text encoding (mostly)

HTTP Request

Method	Path	HTTP version	Headers	Message body
GET	/pszcmg/Project_ideas.html	HTTP/1.1	…	

- **Method**: verb, i.e. nature of request
- **Path**: resource, i.e. object of request
- HTTP **version**
- **Headers**: additional information
  - E.g. “Host: <hostname>” (from URL, required), authentication information, content type options, …
- **Body**: (optional) request data
  - E.g. document being uploaded

HTTP methods

- HTTP defines standard types of request (verbs)
  - **GET**: requests a document; server sends a copy of the document
    - GET is the normal request from a browser
  - **HEAD**: request status information; server sends status (headers) only for the requested document
  - **POST**: sends data to a server, e.g. for form submission; server appends (adds) the data
  - **PUT**: also sends data; server replaces the data
  - **DELETE**: request document be deleted
  - OPTIONS: query which methods, etc., can be used
  - (Others: TRACE, CONNECT)

HTTP Response

HTTP version	Status code	Reason	headers	Message body
HTTP/1.1	200	OK	…	Resource data

- **Status codes** include:
  - 200 OK; 400 Bad Request; 403 Forbidden; 404 Not Found; 301 Moved; 307 Redirect; …
    - (Reason is text description of code)
- Common **headers** include:
  - **Content-Length** (in bytes)
  - **Content-Type**, e.g. is it HTML? (“text/html”, a MIME type)

For example… (raw request)

Try it!
See lab 5

- Try, at a command line prompt:

```
wget -d https://people.cs.nott.ac.uk/pszcmg/Project_ideas.html
```

or

```
curl -v https://people.cs.nott.ac.uk/pszcmg/Project_ideas.html
```

or try the debug/developer mode in your browser

CRLF after each line

```
GET /pszcmg/Project_ideas.html HTTP/1.1
User-Agent: Wget/1.14 (linux-gnu)
Accept: */*
Host: people.cs.nott.ac.uk
Connection: Keep-Alive
```

Blank line

Don't worry about all of the details

For example… (raw response)

```
HTTP/1.1 200 OK
Date: Mon, 06 Oct 2014 10:42:58 GMT
Server: Apache/2.4.6 (Linux/SUSE)
Last-Modified: Tue, 08 Apr 2014 13:21:57 GMT
ETag: "1b0b-4f687de5016d4"
Accept-Ranges: bytes
Content-Length: 6923
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Content-Type: text/html

<html><head>
…
```

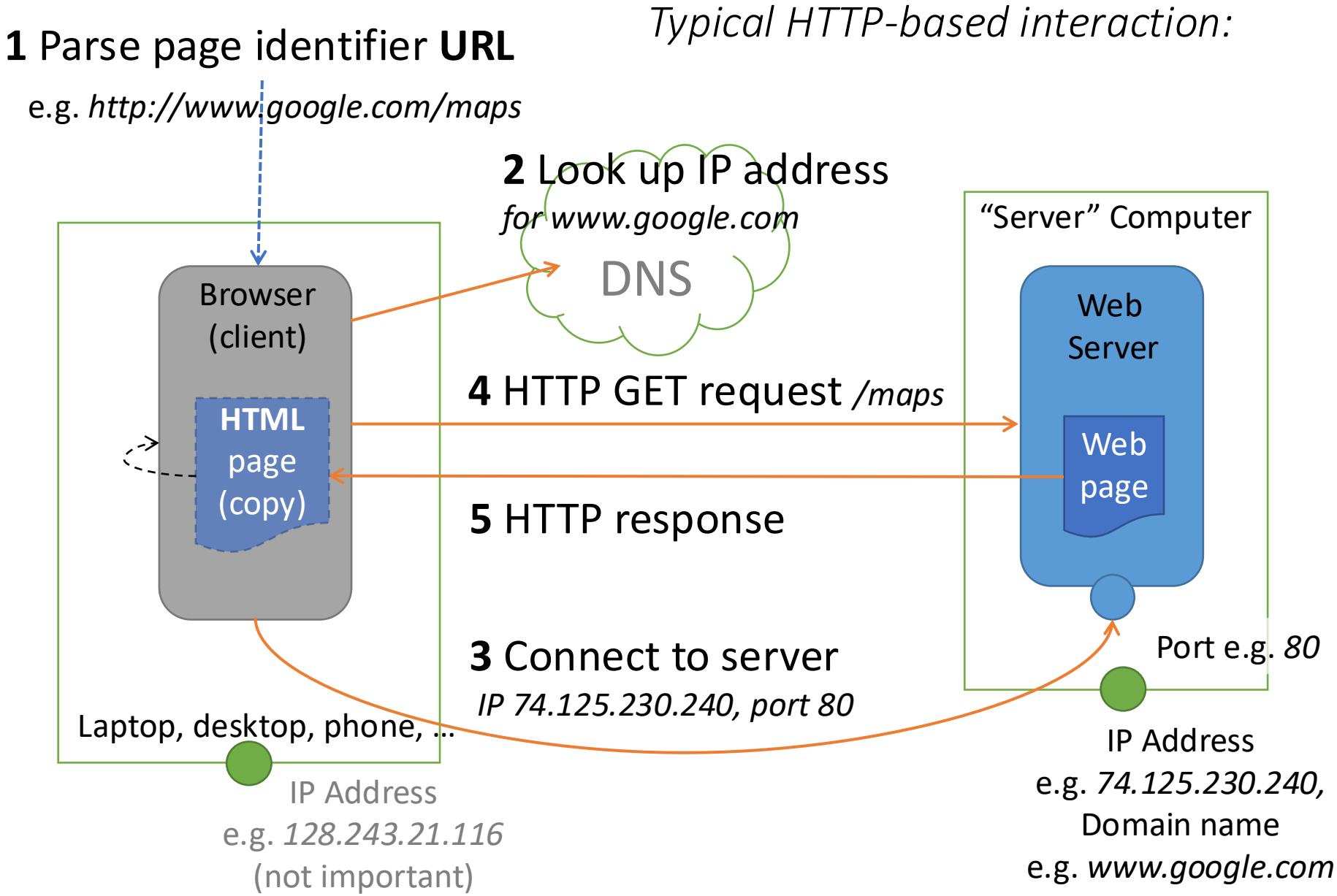
CRLF after each line

Blank line

The actual file – 6923 bytes of HTML

Don't worry about all of the details

Typical HTTP-based interaction:



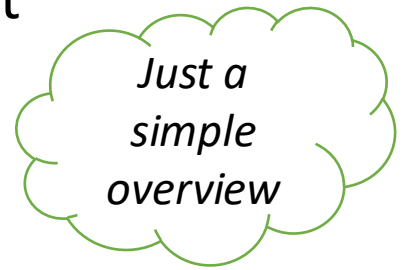
Discussion

- HTTP doesn't care *how* responses are generated
- HTTP doesn't care what format/encoding the data is in
  - Described by Content-type header, e.g. HTML, image, Word, PDF, …
- HTTP doesn't just transport [passive] "data"
  - E.g. modern web pages include many Javascript (i.e. code) elements - see next section
- HTTP has a secure variant, HTTPS, which inserts a secure communication protocol (TLS) on top of TCP
  - => encrypted data transfer (using private key cryptography) and authentication of the server (hostname or IP address) (using public key cryptography)

HTML

Document Representation with HTML

- **HyperText Markup Language** (**HTML**) is a representation standard that specifies the **syntax*** of a web page for display in a browser
- HTML has the following general characteristics:
  - Uses a textual representation
  - Describes pages that contain **multimedia**
  - Permits a **hyperlink** to be embedded in an arbitrary object
  - Allows a document to include **metadata**



*and some aspects of semantics, i.e. meaning, e.g. “heading”, “em(phasis)”

Document Representation with HTML

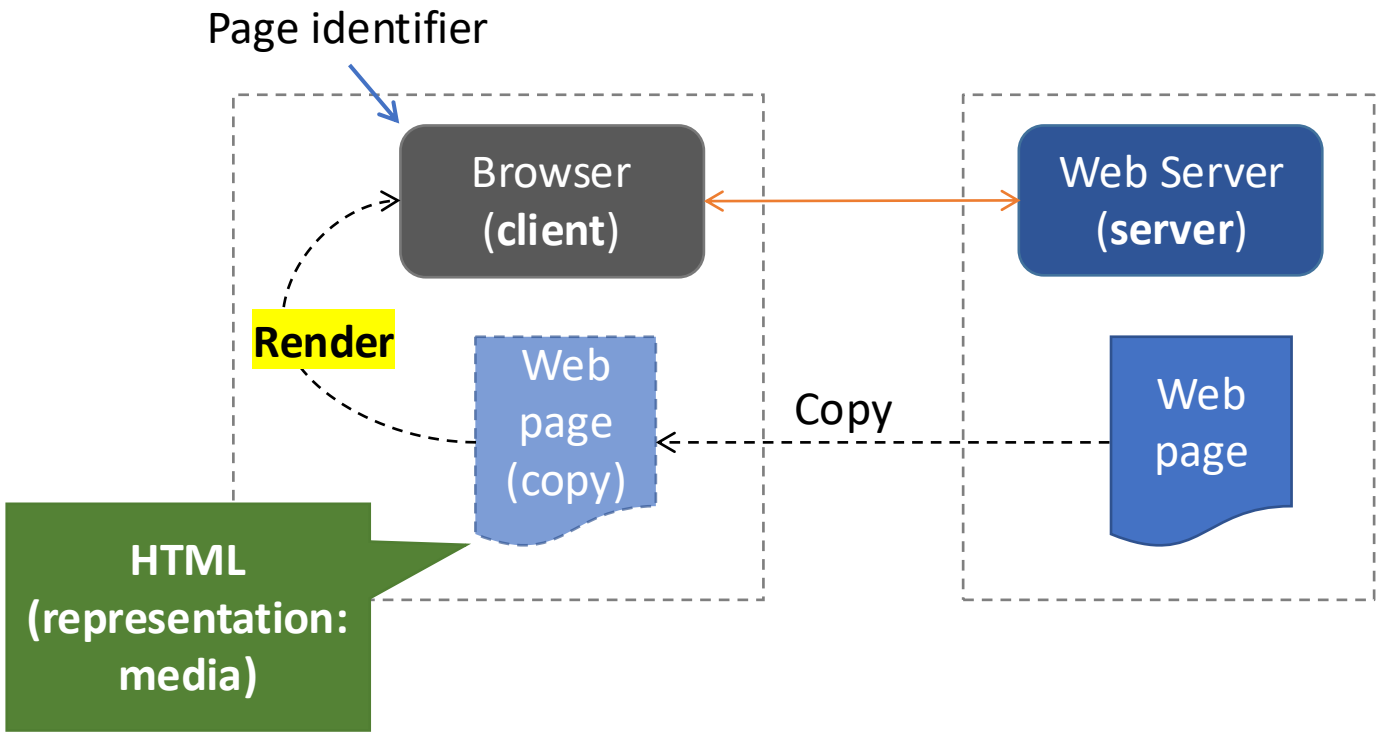
```
<html>
  <head>
    <title>The title</title>
  </head>
  <body>
    <h1>Hello!</h1>
    <a href="doc1">get doc 1</a>
  </body>
</html>
```

Try the “view [page] source” option in a web browser

This module does not care about the details of HTML tags, etc.

The HTML can include <a> elements which specify a hyperlink to another web page using a URL

Recap: viewing a web page



HTTP Servers

HTTP server ports

- HTTP servers normally run on **well-defined ports**
  - http: port 80, https: port 443
  - So all TCP connection requests to a host will (initially) arrive at the same process
  - So TCP port numbers (layer 4) are NOT normally useful for distinguishing between services using HTTP on a single host
    - Even if the host has more than one domain name they usally all map to the same IP address
- [Aside: custom HTTP servers often run on different ports by default, and (for example) Docker's NAT is used to map the standard host ports (80, 443) to one specific container & port]

HTTP server configuration

Q: Anyone set up their own web server? What for?

- Each server will require typically explicit configuration, e.g.
  - Runtime settings, e.g. numbers of threads & connections
  - Which port(s) to bind and which protocol(s) to use
  - Security configuration, e.g. HTTPS certificates
  - How to respond to errors
  - What logs to generate
  - **How to map request URLs to files/folders**
  - **How to map request URLs to other services**
- E.g.
  - https://nginx.org/en/docs/example.html
  - https://httpd.apache.org/docs/2.4/vhosts/examples.html

Routing Requests

- A server can handle each request differently depending on, e.g., its:
- URL domain (host) name
  - A.k.a. **virtual hosts** – several domain names can refer to the same host/IP address but get individual responses
- URL path
  - A.k.a. path **filtering** or **routing**
- Identified file type/extension
  - E.g. execute ".php" files instead of copying them back
- Client IP address
  - E.g. for **load balancing** resource-intensive requests over multiple servers – see later
- HTTP Headers
  - E.g. authentication, body content type or size, if-not-modified

NB all use standard information carried in the HTTP request (or TCP connection)

Handling Requests: common options

- **Static file** serving – the web server uses the URL path to identify a file which is copied back as the response body
- **Reverse proxy** – the original server becomes an HTTP client to pass the request onto another web server
  - Which may be on a different port and/or host
- **CGI** – the server executes a program (identified by the URL) in a separate process to handle the request
- **Dynamically loadable module** – the server loads a runtime engine to execute (e.g.) a file in a scripting language (identified by the URL)
  - E.g. PHP in Apache via mod_php
- **Custom web server** written using a language-specific web server framework - the web server includes the code to handle specific requests, e.g.
  - Node.js Express, python bottle, Django, …
  - **Java Servlets** – the (Java Enterprise) server (e.g. Tomcat) uses dynamic code loading and configuration files or code annotations to find a specific Java object to call for each request – see Lab 6

Q: Anyone written their own web application? How?

Challenges for Distributed Systems (revisited – see Lecture 2) & the WWW

Introduction to Challenges

- Lots of research focusses on addressing one or more of these challenges
  - e.g. reliability, performance (quality of service), security
- Projects and organisations often adopt a position on some of these challenges,
  - i.e. how much they care about them,
- E.g. the WWW prioritises **openness** & **heterogeneity**…

Heterogeneity

- Recap: Coping with system component variability…
  - C.f. Java Data Input/Output Streams
- E.g. the WWW depends on common protocols (HTTP) and encodings (URLs and HTML) that are independent of programming language, CPU architecture, etc.

Failure Handling
(see also later lectures on Failure & Reliability)

- Recap: Coping with partial failure…
  - C.f. simple TCP server
    - Deploy multiple servers?
    - Retrying failed connections?
- E.g. the WWW is loosely coupled, so that if one web server fails other clients and servers are not affected

Concurrency
(see also Operating Systems and Concurrency module)

- Recap: correctness and performance with concurrency…
  - C.f. the example TCP server is multi-threaded so that it can serve multiple **independent** clients concurrently
- E.g. most web servers will have to cope with many concurrent requests are still give correct responses

Openness

- In general we want to be able to extend, re-implement and inter-operate with distributed systems
  - Unless as a vendor we want to “lock” customers into our own proprietary solution
- Usually this means agreeing and publishing the protocols and interfaces used,
  - e.g. the Internet Protocols
- E.g. the WWW uses standard published protocols, so that any web client can interact with any web server
  - Any organisation can contribute to WWW standards via IETF/W3C
  - (It can also support new content types and services because it doesn’t specify how servers work internally to handle requests)

Security

- Many of the resources in distributed systems are valuable
- A range of security measures may be needed,
  - including authentication, access control, audit, encryption and firewalls
- E.g. the WWW has a secure version of HTTP called HTTPS, which
  - encrypts data transfers and
  - checks the identity of the server

Scalability

- A system is **scalable** if it continues to work well as it grows
  - E.g. in 1993 there were less than 2 million computers on the Internet,
  - in 2003 there were about 200 million and
  - today there are estimated to be over 10 billion phones/computers and 30 billion other devices connected to the Internet
- Requires careful design and engineering,
  - For example to avoid bottlenecks or single points of failure.
  - The Internet and DNS are good examples
- E.g. web browsers cache fetched content to reduce subsequent data transfers, i.e. network and server load

Transparency

- I.e. users/other systems can't "see" the internal details of the system
- For example:
  - **Access** transparency – local vs remote (e.g. RPC/RMI)
  - **Location** transparency – where exactly?
  - **Replication** transparency – using replicas
  - **Failure** transparency – hiding partial failures
  - **Mobility** transparency – unaffected by physical movement
- E.g. with URLs support access & location transparency
  - you don't tend to think about exactly where a web page comes from or whether it is local or remote

Quality of Service

- Various non-functional aspects, including performance, reliability and security
  - For example, (how) can you build a reliable service on an unreliable network?
  - How can you automatically adjust streamed video quality to cope with variable bandwidth?
- E.g. HTTP is normally as reliable as TCP (i.e. it fails if the underlying TCP connection fails), but there are extensions for reliable HTTP
  - E.g. to uniquely identify and retry HTTP requests

The World Wide Web and URLs - Summary

- The **World Wide Web** (WWW) is an **open** distributed system based on standardized **protocols** and **representations**
  - Mainly comprising **Browsers** (clients) are **Web Servers** (servers)
- A **Uniform Resource Locator** (**URL**) specifies a web page
  - **protocol**://**computer_name**:**port**/**document_name**?**parameters**
- The client (browser) uses the protocol, computer_name and port to connect to a particular server and request the document
  - Usually the protocol is **http:** (so it uses **HTTP**)
  - It then displays the returned content (or error)

HyperText Transfer Protocol (HTTP) Summary

- **HyperText Transfer Protocol** (HTTP) is the protocol that a **client** (e.g. browser) uses to interact with a web **server**
- Textual **request-response** protocol
- Defines a standard set of request methods:
  - **GET**, **HEAD**, **POST, PUT, DELETE, OPTIONS** (…)
- Uses Headers to pass extra information
  - E.g. “Content-Type”, “Content-Length”
- Defines a standard set of **status** (response) **codes**, e.g.
  - 200 OK; 400 Bad Request; 403 Forbidden; 404 Not Found; 301 Moved; 307 Redirect; …
- Can transfer content of any type or size of data each way

HTTP Server Summary

- HTTP servers typically require explicit configuration
- HTTP servers use a range of strategies to **map** request URLs to specific request handlers
  - E.g. mapping based on “Host:” header => virtual hosts
    - i.e. different content based on the DNS name used, even with the same IP address
- HTTP servers can handle requests in various ways
  - E.g. static file serving, CGI, reverse proxy, loadable module, custom code

Challenges for Distributed Systems Summary

- **Heterogeneity** – coping with system component variability
- **Failure Handling** – coping with partial failure
- **Concurrency** – correctness and performance with concurrency
- **Openness** – to extension or reproduction
- **Security** – protection from threats
- **Scalability** – good performance as systems grow
- **Transparency** – selective abstraction
- **Quality of Service** - performance & other non-functional characteristics

Next steps

- Lab 5: basic HTTP and browsers
- Lecture 9: more about HTTP and using HTTP between programs
  - i.e. Web APIs
  - Lab 6: Java HTTP